

SecurePasswordManager

Seminar Report for COMP.SEC.300 Secure Programming

Course: Secure Programming (COMP.SEC.300)

Author: Joni Salonen

Technology Stack: C#, .NET 8.0, WPF, AES-256-GCM, Argon2id, CSPRNG

Date: May 2026

Focus: Cryptographic Controls, Secure Randomization, Threat Mitigation

Abstract

This report documents the design, implementation, and security analysis of SecurePasswordManager, a cryptographic password storage application. The application implements AES-256-GCM for authenticated encryption, Argon2id for memory-hard key derivation, and CSPRNG for password generation. The system enforces input validation, multi-layered access control, and minimizes plaintext exposure. Comprehensive unit testing validates cryptographic operations. Results demonstrate secure programming principles applied to password management.

1. Introduction

Password managers are critical infrastructure for security. A secure password manager must address: strong password generation, encrypted storage, secure key derivation, input validation, authentication/authorization, plaintext minimization, and secure import/export.

Security Requirements

- Strong Password Generation: CSPRNG with configurable entropy
- Encrypted Storage: AES-256-GCM with integrity verification
- Secure Key Derivation: Argon2id resistant to GPU/ASIC attacks
- Input Validation: Reject malformed data before processing
- Authentication & Authorization: Master password, unlock state, access guards
- Plaintext Minimization: Encrypted at-rest, limited view duration
- Secure Import/Export: Validation, duplicate detection, integrity checking

2. Research Question and Objectives

Research Question

"How can a password manager be designed to demonstrate practical secure programming principles including cryptographically secure random generation, authenticated encryption, secure key derivation, and threat-aware access controls?"

Objectives

1. Implement CSPRNG-based password generation with configurable character classes and entropy validation
2. Design AES-256-GCM authenticated encryption with unique IV per entry and AAD binding
3. Integrate Argon2id key derivation with memory-hard parameters
4. Implement comprehensive input validation at entry form and import functions
5. Enforce multi-layered access control: master password, unlock state, per-entry guards
6. Minimize plaintext exposure through encryption, modal reveals, secure memory clearing
7. Implement secure import/export with validation and duplicate detection
8. Create unit tests covering password generation, encryption, key derivation, and access control

3. Methodology

Phase 1: Design & Threat Modeling

Purpose: Identify attack surfaces, security requirements, and algorithm selection.

Phase 2: Implementation of Cryptographic Controls

Purpose: Implement AES-GCM, Argon2id, CSPRNG with secure APIs and error handling.

Phase 3: Access Control & Input Validation

Purpose: Implement master password auth, unlock state management, and input validation.

Phase 4: Comprehensive Testing & Validation

Purpose: Create unit tests for password generation, encryption, key derivation, import/export, and access control.

Phase 5: Security Analysis & Documentation

Purpose: Analyze test results against NIST/OWASP guidelines and document limitations.

4. Background and Secure Programming Principles

4.1 Cryptographically Secure Random Number Generation (CSPRNG)

System.Random and predictable generators violate CWE-330. SecurePasswordManager uses RandomNumberGenerator, backed by OS CSPRNG (CryptGenRandom on Windows, /dev/urandom on Linux).

Password Policy and Entropy

Minimum Length: 8 characters

Maximum Length: 128 characters

Recommended Entropy: 80+ bits (NIST SP 800-63B baseline)

Character Pool Sizes: 92 chars (uppercase + lowercase + digits + symbols)

32-char password from 92-char set: Entropy ≈ 209.5 bits (Very High)

4.2 Authenticated Encryption: AES-256-GCM

AES-256-GCM provides confidentiality and integrity. GCM includes authentication tag that detects tampering. This mitigates CWE-327 and CWE-353.

Algorithm: AES-256 with GCM mode

Key Size: 256 bits (32 bytes)

Nonce/IV Size: 12 bytes (96 bits)

Authentication Tag: 16 bytes (128 bits)

Associated Data (AAD): Service name and entry ID bound to ciphertext

4.3 Memory-Hard Key Derivation: Argon2id

Argon2id is recommended by OWASP for password storage. It is memory-hard, making GPU/ASIC attacks expensive. This mitigates CWE-916.

Memory Cost: 65,536 KB (64 MB)

Time Cost (Iterations): 3 passes

Parallelism: 4 lanes

Hash Output Length: 32 bytes (256 bits)

Brute-force Resistance: 10-char password: 2^{60} search space ≈ 36 million years at 1 guess/sec

4.4 Input Validation and Sanitization

Input validation is primary defense against injection attacks. CWE-20 is most common in OWASP Top 10. SecurePasswordManager validates at multiple entry points.

Entry Form: Service name (non-empty, max 255 chars), Username (max 255 chars), Password (8-128 chars)

Import Function: Structure validation, data type checking, duplicate detection

Unlock Operation: Master password format (non-empty string)

4.5 Authentication and Multi-Layer Access Control

The application enforces authentication and authorization:

Layer 1: Master Password: Unlock vault by deriving key from master password

Layer 2: Unlock State: VaultService tracks whether vault is unlocked

Layer 3: Per-Entry Operations: View/Edit/Delete operations guarded by unlock state

4.6 Minimizing Plaintext Exposure

The application implements strategies to limit plaintext exposure:

At-Rest Encryption: All passwords encrypted with AES-256-GCM before storage

In-Memory Clearing: Sensitive data cleared after use

View-Action Modal: Passwords revealed only in focused modal dialog

5. System Design and Implementation Overview

Project Structure

SecurePasswordManager is organized into three .NET projects:

SecurePasswordManager.Core: Cryptographic primitives, business logic, data models

SecurePasswordManager.App: WPF user interface and UI state management

SecurePasswordManager.Tests: Automated xUnit test suite

Key Technologies

AES-256-GCM: Authenticated encryption for password storage

Argon2id: Memory-hard key derivation

RandomNumberGenerator: OS-backed CSPRNG

CSV/JSON Parsing: Import/export with validation

WPF (.NET 8.0): Cross-platform UI framework

xUnit: Testing framework

6. Key Implementation Details

6.1 Password Generation with CSPRNG

Password generation is implemented with the following algorithm:

9. Validate requested length against policy bounds
10. Build character pool from selected character classes
11. Optionally exclude ambiguous characters (0, O, 1, l)
12. Calculate entropy: $E = \text{length} \times \log_2(\text{pool_size})$
13. Reject if entropy < minimum threshold
14. Use rejection sampling with `SecureRandom.GetBytes()`
15. Shuffle password array using Fisher-Yates algorithm

6.2 AES-256-GCM Encryption Implementation

Each password entry is encrypted with:

Plaintext: Service name, username, password, notes

Key: 32 bytes derived from master password

Nonce/IV: 12 bytes generated fresh per entry

Associated Data (AAD): Entry ID and service name

Output: [Version] + [Nonce] + [Encrypted Data] + [Auth Tag]

6.3 Argon2id Key Derivation

Master password → AES-256 key derivation:

Input: Master password (plaintext string)

Salt: 32 bytes from `RandomNumberGenerator`

Algorithm: Argon2id with tuned parameters

Output: 32 bytes (256 bits)

Time: ~500ms per derivation

6.4 Import and Export with Validation

The `ImportService` validates:

CSV Structure Validation: Checks for required columns

Data Type Checking: Ensures fields are strings

Duplicate Detection: Warns of duplicate entries

Master Password Protection: Encrypts imported entries

6.5 UI Access Control Enforcement

VaultWindow enforces unlock state checks:

Locked Vault: View/Edit/Delete buttons disabled

Unlock Operation: Master password verified, key cached

Unlocked Vault: Actions enabled, plaintext revealed in modal

Lock Operation: Key cleared, UI returns to locked state

7. Security Testing and Validation

Comprehensive unit tests validate security properties:

Test Case	Security Purpose	Threat Mitigated
Password generation returns correct length	Validates entropy calculation	CWE-330: Insufficient randomness
Generated passwords contain required character classes	Ensures diversity	CWE-330: Weak patterns
Entropy below minimum is rejected	Validates entropy threshold	CWE-326: Inadequate strength
Ambiguous characters excluded on demand	Validates character filtering	CWE-20: Input validation
Encryption/decryption round-trip succeeds	Validates AES-GCM implementation	CWE-327: Broken encryption
Tampering with ciphertext detected	Validates authentication tag	CWE-353: Missing integrity
Argon2id key derivation is deterministic	Same input produces same key	CWE-916: Weak derivation
Import rejects malformed CSV	Validates CSV structure	CWE-20: Input validation
Import detects duplicate entries	Prevents reimporting	Business consistency
UI access control blocks locked vault operations	Validates unlock state enforcement	CWE-862: Improper access
View action requires unlock state	Plaintext reveal only when authorized	CWE-522: Session expiration

8. Results and Security Analysis

8.1 Password Generation Quality Results

Unit tests confirm:

- Generated passwords achieve 60+ bits entropy for default passwords
- Default 32-char passwords achieve ~209 bits entropy
- Character class validation confirms all required classes present
- Ambiguous character filtering works correctly
- Session settings persist across multiple generation calls

8.2 Encryption Integrity and Tampering Detection

AES-GCM demonstrates:

- Round-trip encryption/decryption with no data loss
- Single byte modification of ciphertext causes authentication failure
- Modifying AAD causes authentication to fail
- Unique nonce generation with no reuse observed

8.3 Key Derivation Resilience

Argon2id analysis:

- Deterministic output: identical keys from same input
- Memory-hard: each derivation requires ~64 MB RAM
- Brute-force: 10-char password = 36 million years at 1 guess/sec
- User experience: ~500ms per unlock (acceptable)

8.4 Input Validation and Import Effectiveness

Validation tests confirm:

- Malformed CSV rejected with error reporting
- Duplicate entries detected with user warning
- Empty fields handled gracefully
- Invalid JSON syntax rejected

8.5 Access Control Enforcement

Multi-layer access control verified:

- Locked vault: entries visible but not decryptable
- Unlock operation: key derived and cached
- Unlocked vault: plaintext password revealed in modal
- Lock operation: key cleared, UI returns to locked state

9. Discussion and Limitations

Strengths

- Strong Password Generation: CSPRNG generates high-entropy passwords
- Authenticated Encryption: AES-256-GCM with per-entry nonces
- Memory-Hard Derivation: Argon2id makes offline brute-force expensive
- Comprehensive Input Validation: Multiple entry points validated
- Multi-Layer Access Control: Master password, unlock state, per-entry guards
- Extensive Unit Testing: 50+ test cases covering security-critical operations
- OWASP/NIST Alignment: Follows guidelines and CWE mitigations

Limitations and Future Work

- Plaintext Reveal Not Hardened: No re-authentication or timeout required
- Plaintext Export Available: CSV export includes plaintext passwords
- No Audit Logging: No persistent audit trail of reveal events
- Master Password Caching: No automatic session timeout implemented
- No Integration Tests: Only unit tests, not end-to-end workflows
- Argon2id Parameters Not Configurable: Hardcoded parameters
- No MFA UI Integration: MFA logic exists but not fully integrated
- Single-Platform Deployment: WPF only runs on Windows

10. Conclusion

This project demonstrates secure programming principles through SecurePasswordManager. The application answers the research question with concrete evidence: AES-256-GCM for authenticated encryption, Argon2id for memory-hard key derivation, RandomNumberGenerator for CSPRNG password generation, comprehensive input validation, multi-layer access control, and secure memory clearing. Security patterns include nonce uniqueness, associated data binding, rejection sampling, unlock state tracking, and plaintext minimization. 50+ unit tests validate cryptographic correctness and access control enforcement. The project demonstrates that secure programming is achievable through careful algorithm selection, rigorous input validation, defensive coding, and comprehensive testing. Future work includes plaintext reveal hardening, audit logging, integration tests, and encrypted export support.

11. Appendix: Verification Checklist

Step-by-step verification checklist:

16. **Build and Compilation:** dotnet build completes with no errors
17. **Unit Test Execution:** dotnet test runs all tests; all pass
18. **Create Vault:** Launch app and create vault with master password
19. **Password Generation Options:** Generate password with custom options
20. **Lock/Unlock Workflow:** Unlock vault with master password
21. **View Password:** Click View on entry; plaintext revealed in modal
22. **Export Passwords:** Export to CSV; file contains all entries
23. **Import and Duplicates:** Import exported CSV; duplicate warning appears
24. **Access Control:** Verify buttons disabled when locked, enabled when unlocked
25. **Test Coverage:** Review CryptoTests.cs for comprehensive test cases